Algorithms

Lecture 3: Insertion Sort

Demo Lecturer

Semester 2, 2026

The Sorting Problem

Input a sequence of n items $A[0..n-1]$

Output the same items rearranged so that $A[0] \le A[1] \le \cdots \le A[n-1]$

Two properties we will ask about every sorting algorithm:

- **in-place**: uses only $O(1)$ extra memory beyond the array
- **stable**: equal keys keep their original relative order

The Idea: Sort a Hand of Cards

Pick up cards one at a time. The cards already in your hand are always sorted.

- Take the next card
- Slide it left past every card that is larger
- Drop it into the gap

In an array: $A[0..i-1]$ is the sorted "hand"; $A[i]$ is the next card.

Insertion Sort: Pseudocode

```
function InsertionSort(A, n)
    for i <- 1 to n - 1
        v <- A[i]
        j <- i - 1
        while j >= 0 and A[j] > v
            A[j + 1] <- A[j]
            j <- j - 1
        A[j + 1] <- v
```

The basic operation is the comparison `A[j] > v`.

Trace on $[5, 2, 4, 6, 1, 3]$

After round i	Array (sorted prefix before the bar)
start	$5 \mid 2\ 4\ 6\ 1\ 3$
$i = 1$	$2\ 5 \mid 4\ 6\ 1\ 3$
$i = 2$	$2\ 4\ 5 \mid 6\ 1\ 3$
$i = 3$	$2\ 4\ 5\ 6 \mid 1\ 3$
$i = 4$	$1\ 2\ 4\ 5\ 6 \mid 3$
$i = 5$	$1\ 2\ 3\ 4\ 5\ 6$

Why It Is Correct

Loop invariant

At the start of the round for index i, $A[0..i-1]$ contains the original first i items in sorted order.

- **Initially** ($i = 1$): a single item is sorted
- **Each round**: the inner loop shifts larger items right and inserts v, so $A[0..i]$ is sorted
- **At the end** ($i = n$): the whole array is sorted

Complexity and Properties

Case	Comparisons
Best (already sorted)	$n - 1 \in \Theta(n)$
Worst (reverse sorted)	$\frac{n(n-1)}{2} \in \Theta(n^2)$

In-place: yes. Stable: yes (we stop shifting at $A[j] \leq v$, never at equal keys).

Summary and Next Time

- Insertion sort grows a sorted prefix; $\Theta(n^2)$ worst case, $\Theta(n)$ best case
- Good on small or nearly sorted inputs
- Next lecture: divide-and-conquer and mergesort